Agentisk triagering av supportärenden

– En teknisk artefakt för säker och regelstyrd kundtjänst

Vetenskaplig rapport i kursen Tillämpning av AI-agenter i Unity 2026

Christian Vittaniemi
Ronnie Samson

VT 2026:C1TAI1

UNIVERSITY
OF BORÅS

Svensk titel: Agentisk triagering av supportärenden: En teknisk artefakt för säker och regelstyrd kundtjänst

Engelsk titel: Agentic Triage of Support Tickets: A Technical Artifact for Safe Rule-Governed Customer Service

Utgivningsår: 2026

Författare: Christian Vittaniemi, Ronnie Samson

Handledare: Johannes Sahlin

Abstract

[Background] As e-commerce platforms increasingly adopt artificial intelligence to handle customer support, the unreliability of Large Language Models (LLMs) poses a significant challenge. Unconstrained LLMs often hallucinate policies or bypass strict business rules when processing complex requests.

[Aim] This report investigates how an agentic workflow can be designed and implemented for reliable automated triage of support tickets.

[Method] Using Design Science Research, a software artifact was developed utilizing a local LLM (Llama 3.1). The architecture minimizes technical complexity while maximizing determinism through three core mechanisms: atomic prompts (micro-steps) to force explicit tool use, isolated context management to prevent distraction from irrelevant history, and deterministic mechanical guards (guardrails) for independent rule validation.

[Results] The artifact was evaluated against 12 predefined support tickets and compared with a simple rule-based baseline. Both approaches produced 7 tickets without validation remarks and 5 tickets flagged by the mechanical guardrail. However, the agentic workflow differed from the baseline by using an iterative process with an average of 1.5 tool calls and 4.0 recorded steps per ticket, compared with 0.0 tool calls and 1.0 step for the baseline. The results therefore primarily demonstrate improved traceability and explicit agentic behaviour, rather than a higher validation pass rate in this limited test set.

[Conclusion] The study concludes that reliable AI-driven triage should combine the natural language capabilities of an LLM with traditional software structures such as explicit tool use, trajectory logging and deterministic validation. In the tested scenarios, the architecture improved transparency and control, while the mechanical guardrail remained necessary for identifying cases requiring manual review or additional information.

[Keywords] Agentic Workflows, Large Language Models (LLM), Customer Support, Triaging, Design Science, Guardrails(Mechanical guard), ReAct.

Sammanfattning

[Bakgrund] I takt med att e-handelsplattformar i allt högre grad integrerar artificiell intelligens för kundsupport, utgör den bristande pålitligheten hos stora språkmodeller (LLM) en betydande utmaning. Obegränsade språkmodeller tenderar ofta att hallucinera fram policyer eller ignorera strikta affärsregler vid hantering av komplexa ärenden.

[Syfte] Denna studie undersöker hur ett tillförlitligt agentiskt arbetsflöde kan designas för automatiserad triagering av supportärenden.

[Metod] Genom ramverket Design Science Research utvecklades en mjukvaruartefakt driven av en lokal språkmodell (Llama 3.1). Arkitekturen är designad för att minimera teknisk komplexitet och maximera determinism genom tre centrala mekanismer: atomära prompter (mikrosteg) för att tvinga fram explicita verktygsval, isolerad kontexthantering för att förhindra distraktion från irrelevant historik, samt deterministiska mekaniska vakter (guardrails) för oberoende regelvalidering.

[Resultat] Artefakten utvärderades mot 12 fördefinierade supportärenden och jämfördes med en enkel regelbaserad baseline. Både baseline och det agentiska arbetsflödet hade 7 ärenden utan validatoranmärkning och 5 ärenden som flaggades av den mekaniska vakten. Skillnaden låg framför allt i processens struktur: baseline utförde enkel enstegsbaserad routing utan verktygsanrop, medan det agentiska arbetsflödet i genomsnitt använde 1,5 verktygsanrop och 4,0 registrerade steg per ärende. Resultatet visar därmed främst förbättrad spårbarhet och tydligare agentiskt beteende, snarare än högre validator-pass rate i den begränsade testmängden.

[Slutsats] Studien drar slutsatsen att tillförlitlig AI-driven triagering bör kombinera språkmodellens textförståelse med traditionella programvarustrukturer såsom explicit verktygsanvändning, trajectory-loggning och deterministisk validering. I de testade scenarierna bidrog arkitekturen främst till ökad transparens och kontroll, medan en regelbaserad guardrail var avgörande för att identifiera ärenden som kräver komplettering eller manuell granskning.

[Nyckelord] Agentiska arbetsflöden, Stora språkmodeller (LLM), Kundsupport, Triagering, Design Science, Mekaniska vakter, ReAct.

1 Inledning 1
1.1 Bakgrund 2
1.2 Forskningsöversikt 2
1.3 Problemdiskussion 2
1.4 Problemformulering 3
1.5 Syfte och forskningsfråga 3
1.6 Målgrupp för arbetet 4
2 Litteratur 5
Iterativt resonerande i målinriktade dialoger (Task-Oriented Dialogue) 5
Behovet av kontextuell begränsning och externa valideringar 5
Känslighet för brus och kontextuell isolering 6
Deterministiska skyddsmekanismer och regelefterlevnad 6
Verktygsanvändning och atomära beslut 6
3 Metod 7
3.1 Forskningsmetod 7
3.2 Ramverk för utvecklingsarbetet 7
3.3 Metodreflektion 8
Redogörelse för användning av AI-verktyg 8
4 Genomförande 9
4.1 Metoder 9
4.2 Test 9
4.3 Resultat 10
4.4 Validering 11
4.5 Måluppfyllelse 11
5 Analys 13
6 Diskussion och slutsatser 14
Slutsatser 14
7 Referenser 16

1 Inledning

1.1 Bakgrund

Kundsupport utgör en kritisk och resurskrävande funktion för e-handelsplattformar, inte minst inom nischade branscher som dykutrustning där ärenden ofta kräver specifik domänkunskap och noggrann hantering. Traditionella, regelbaserade chattbottar brister ofta i flexibilitet och förmåga att förstå nyanser i kundernas problem. Samtidigt riskerar oinskränkta stora språkmodeller (LLM), som genererar hela svar i ett enda svep, att hallucinera information eller ignorera kritiska affärsregler när de ställs inför komplexa supportärenden.

För att överbrygga klyftan mellan statiska regler och opålitlig textgenerering har intresset riktats mot agentiska arbetsflöden. I ett sådant system agerar språkmodellen som en resonerande motor som stegvis utvärderar information, använder externa verktyg vid behov och sparar historik för att fatta välgrundade beslut. Det skapar dock nya tekniska utmaningar kring hur man bäst strukturerar och begränsar dessa agenter för att säkerställa att de arbetar tillförlitligt, vilket gör designen av arkitekturen till ett centralt problemområde.

1.2 Forskningsöversikt

Tidigare forskning inom tillämpad artificiell intelligens har visat att stora språkmodeller presterar avsevärt bättre på komplexa uppgifter när de ges möjlighet att kombinera stegvist resonerande med förmågan att integrera med externa verktyg, ett koncept som ofta benämns som ReAct (Reasoning and Acting). Studier belyser dock en inbyggd problematik: agenter som ges för stort kontextuellt handlingsutrymme tenderar ofta att underprestera i målinriktade uppgifter (Elizabeth *m.fl.*, 2025).

För att stävja detta betonar aktuell forskning vikten av en avgränsad och tydlig systemdesign. Det handlar bland annat om att begränsa modellens svarsalternativ genom atomära uppgifter och riktad verktygsanvändning (Schick *m.fl.*, 2023). Samtidigt påvisas vikten av att dynamiskt isolera och rensa modellens kontext för att minska kognitivt brus från tidigare interaktioner (Shi *m.fl.*, 2023). Vidare diskuteras i litteraturen nödvändigheten av oberoende valideringsmekanismer, ofta kallat guardrails. Dessa fungerar som regelbaserade spärrar som tvingar AI-systemets utdata att valideras mot fördefinierade verksamhetskrav innan ett faktiskt beslut verkställs i ett produktionssystem (Rebedea *m.fl.*, 2023).

1.3 Problemdiskussion

Även om agentiska flöden visar stor potential, medför de betydande risker om de implementeras utan strikta ramverk. Om en språkmodell får fria händer att hantera ett supportärende kan den fatta beslut som strider mot gällande affärsregler till exempel genom att godkänna en retur som har passerat 30-dagarsgränsen eller hallucinera fram fiktiva policyer. Det praktiska problemet är därmed inte *om* en LLM kan formulera ett svar till en kund, utan hur man bygger en arkitektur som tvingar modellen att systematiskt följa verksamhetens processer. Detta skapar ett behov av att designa system som bryter ner uppgiften i mikrosteg, tillämpar isolerad kontext och använder externa valideringsmekanismer för att säkra utfallet. Denna studie fokuserar på att överbrygga gapet mellan osäker textgenerering och kravet på deterministisk ärendehantering.

1.4 Problemformulering

Huvudproblemet som detta utvecklingsarbete adresserar är hur man rent tekniskt och arkitektoniskt designar ett agentiskt arbetsflöde som är tillräckligt robust och pålitligt för automatiserad triagering av supportärenden. Den centrala utmaningen ligger i att integrera modellens flexibla textförståelse med en deterministisk regelvalidering, utan att systemet fastnar i loopar eller genererar felaktiga format. För att lösa detta problem antar arbetet ett konstruktivt tillvägagångssätt där en mjukvaruartefakt utvecklas, testas och utvärderas, vilket i sin grundstruktur förhåller sig till principerna inom Design Science.

1.5 Syfte och forskningsfråga

Syftet med denna studie är att designa, utveckla och utvärdera ett agentiskt arbetsflöde för automatiserad triagering av supportärenden. Utvecklingsarbetet fokuserar på att uppnå hög tillförlitlighet genom en medvetet avskalad och pragmatisk systemdesign. Genom att tillämpa tydliga och avgränsade designmönster specifikt atomära prompter för stegvis beslutsfattande och mekaniska vakter för oberoende regelvalidering är målsättningen att bygga ett robust system med minsta möjliga tekniska komplexitet. Studien ämnar utvärdera hur dessa designval påverkar systemets tillförlitlighet och dess förmåga att styra en stor språkmodell (LLM) i en avgränsad supportmiljö.

Forskningsfråga:

Hur kan ett agentiskt arbetsflöde utformas för att tillförlitligt triagera och hantera supportärenden?

> **Delfråga 1:**
> Hur påverkar användningen av atomära prompter agentens förmåga att utföra korrekta verktygsval?
>
> **Delfråga 2:**
> Vilken roll spelar mekaniska vakter (validering) för att säkerställa att LLM-agenten följer uppsatta affärsregler?

1.6 Målgrupp för arbetet

Målgruppen för detta arbete utgörs i första hand av systemutvecklare, systemarkitekter och tekniska projektledare som arbetar med att integrera stora språkmodeller (LLM) i kundnära tillämpningar. Även beslutsfattare och produktägare inom e-handel och kundsupport kan dra nytta av insikterna kring hur agentiska arbetsflöden kan automatisera ärendehantering på ett säkert, förutsägbart och regelstyrt sätt.

2 Litteratur

Detta kapitel redogör för tidigare forskning och teorier som sätter utvecklingen av agentiska arbetsflöden i ett vetenskapligt sammanhang. Motivet till valet av litteratur är att belysa den centrala utmaningen med att integrera generativa språkmodeller i strikta, regelbaserade miljöer, såsom supportsystem. Litteraturen fokuserar därför på forskning kring iterativt resonerande och det teoretiska behovet av externa skyddsmekanismer.

Iterativt resonerande i målinriktade dialoger (Task-Oriented Dialogue)

Ett centralt koncept inom tidigare forskning för att frångå statisk textgenerering och i stället lösa komplexa uppgifter är ReAct (Reasoning and Acting). I empirisk forskning undersöks hur denna strategi tillämpas för målinriktade uppgifter, så kallad Task-Oriented Dialogue (TOD). I dessa system instrueras språkmodeller att generera både resonemang och specifika handlingar växelvis. Studier visar dock på en inbyggd problematik i dessa arbetsflöden: även om ReAct-drivna agenter upplevs som naturliga och ofta formulerar sig mycket självsäkert, tenderar de att underprestera i faktiska simuleringar när det gäller att lösa den underliggande uppgiften korrekt (Elizabeth *m.fl.*, 2025).

Behovet av kontextuell begränsning och externa valideringar

För att stävja den identifierade problematiken kring LLM-agenters underprestation i strikta uppgifter, pekar relaterad forskning på vikten av att begränsa modellernas handlingsutrymme. Litteraturen diskuterar frekvent användningen av skyddsmekanismer (ofta benämnt som guardrails), vilka fungerar som externa valideringssteg. Teorin bakom detta är att eftersom stora språkmodeller är probabilistiska, krävs deterministiska system för att validera modellens utdata innan ett beslut kan betraktas som tillförlitligt. Vidare lyfts vikten av att dynamiskt hantera agentens kontext för att undvika att modellen fastnar i loopar eller drar felaktiga slutsatser baserat på irrelevant historik. Detta utgör den teoretiska grunden för varför agentiska system ofta behöver separera själva textgenereringen från regeltillämpningen (Elizabeth m.fl., 2025).

Känslighet för brus och kontextuell isolering

För att ytterligare hantera de utmaningar som uppstår när språkmodeller ska utföra specifika uppgifter är hanteringen av inmatningskontexten (prompt context) av yttersta vikt. Empiriska studier visar att stora språkmodeller är mycket känsliga för brus och lätt distraheras om de matas med irrelevant historik eller ostrukturerad information. När modeller exponeras för överflödig kontext sjunker deras förmåga att dra korrekta slutsatser, vilket ökar risken för att de frångår sina ursprungliga instruktioner eller hallucinerar fram felaktiga svar. Detta understryker nödvändigheten av att kontinuerligt isolera och rensa den information som modellen har tillgång till vid varje enskilt beslutssteg. Genom att enbart exponera systemet för absolut nödvändig data minimeras det kognitiva bruset, vilket i sin tur hjälper agenten att bibehålla fokus på den aktuella uppgiften (Shi *m.fl.*, 2023).

Deterministiska skyddsmekanismer och regelefterlevnad

En av de största utmaningarna med att implementera LLM-baserade system i verkliga produktionsmiljöer är att garantera att uppsatta affärsregler och säkerhetsprotokoll efterlevs till fullo. Eftersom språkmodeller i grunden bygger på sannolikhetslära, fastslår nyare litteratur inom AI-säkerhet att de måste kompletteras med deterministiska skyddsmekanismer. Dessa fristående system, ofta benämnda som mekaniska vakter eller "guardrails", fungerar som oumbärliga kontrollinstanser. De säkerställer att modellens utdata valideras rigoröst mot ett oföränderligt, fördefinierat regelverk innan ett beslut tillåts verkställas. Teorin understryker att en strikt separation mellan agentens kreativa problemlösning och systemets regeltillämpning är ett krav för att verksamhetskritiska processer inte ska kompromettеras av oväntade modellbeteenden (Rebedea *m.fl.*, 2023)

Verktygsanvändning och atomära beslut

För att öka precisionen och minska risken för att AI-system hallucinerar fram felaktiga lösningar, har integrationen av externa verktyg blivit en central forskningsfråga. Istället för att förlita sig på att modellen ska generera hela svar internt, visar studier att språkmodeller presterar avsevärt bättre när de styrs till att anropa specifika API:er och verktyg i isolerade steg. Genom att pausa textgenereringen för att hämta extern data eller utföra en specifik handling – till exempel via snäva, atomära uppmaningar – säkerställs att besluten baseras på faktiska systemtillstånd snarare än probabilistiska gissningar. Denna metod minskar tolkningsutrymmet och tvingar fram explicita verktygsval framför fritextsvar (Schick *m.fl.*, 2023).

3 Metod

3.1 Forskningsmetod

Den övergripande undersökningsstrategin för denna studie är av konstruktiv och experimentell karaktär, inriktad på att lösa ett praktiskt problem genom systemutveckling. För att besvara forskningsfrågorna har en mjukvaruartefakt utvecklats i form av en LLM-baserad agent för triagering.

Utvecklingen av artefakten genomfördes med en tydlig arkitektonisk strategi för AI-assisterade system. Framtagandet styrdes av strukturerade designmönster, primärt genom tillämpningen av atomära prompter (micro-steps) för att bryta ner beslutsprocesser, samt isolerad kontexthantering för att säkerställa att agenten endast exponerades för strikt relevant och isolerad kontext. Vidare tillämpades en strategi för säker informationsinhämtning genom Read-Only prompter för externa policys, samt mekaniska vakter (mechanical guards) för att framtvinga en deterministisk regelefterlevnad. En mer utförlig och teknisk beskrivning av hur dessa metoder implementerades presenteras i kapitel 4 (Genomförande och resultat).

Utvärderingen av artefakten genomfördes genom strukturerade tester i en avgränsad miljö. Agenten exponerades för en testdatabas bestående av 12 representativa supportärenden. Utöver detta implementerades en enkel regelbaserad baseline i eval/baseline.py. Baseline-flödet utförde direkt routing baserat på ärendetyp och enkla fältkontroller, men saknade agentiska egenskaper såsom verktygsanvändning, policyhämtning, trajectory och iterativt beslutsfattande. Evaluation-runnern i eval/run_eval.py körde både baseline och det agentiska arbetsflödet mot samma ärenden, validerade resultaten med validate_decision och sparade utfallet i eval/evaluation_results.json. Utvärderingen mätte bland annat antal ärenden utan validatoranmärkning, antal ärenden flaggade av den mekaniska vakten, genomsnittligt antal verktygsanrop och genomsnittligt antal registrerade steg per ärende.

3.2 Ramverk för utvecklingsarbetet

Denna studie relaterar starkt till riktlinjerna för Design Science Research (DSR), vilket utgör ramverket för utvecklingsarbetet. Ansatsen är problemdriven och syftar till att skapa, samt utvärdera, en innovativ IT-artefakt – i detta fall ett agentiskt arbetsflöde för triagering av supportärenden. Ramverket har tillämpats genom en iterativ process uppdelad i de två centrala faserna *byggande* (build) och *utvärdering* (evaluate). Under byggfasen designades arkitekturen med ett starkt fokus på atomära prompter och oberoende valideringsmekanismer. Under utvärderingsfasen testades sedan artefakten mot fördefinierade affärsregler och exekveringsscenarier för att empiriskt bedöma dess praktiska nytta och tillförlitlighet i relation till den identifierade problemformuleringen.

3.3 Metodreflektion

Att använda Design Science som ramverk visade sig vara ett lämpligt val, då det möjliggjorde en praktisk demonstration av de arkitektoniska mönstren snarare än enbart teoretiska resonemang. Genom att rent praktiskt utveckla artefakten och implementera tekniker som

isolerad kontexthantering via trajectory-loggen och read-only prompter för policydokument, kunde agentens beteende utvärderas under realistiska förutsättningar.

En identifierad begränsning med den valda metoden är dock utvärderingens omfattning. Utvärderingen genomfördes i en kontrollerad testmiljö med en avgränsad mängd fördefinierade supportärenden (JSON-data). För att fullt ut validera systemets robusthet mot oändliga loopar eller oförutsägbara fel hade tester krävts med en betydligt större volym av ostrukturerad, verklig användardata. Trots denna begränsning bedöms metoden ha genererat ett tillräckligt och relevant underlag för att besvara studiens forskningsfrågor och illustrera hur mekaniska vakter kan säkra agenters beteende. Denna reflektion presenteras här för att tydliggöra metodvalets styrkor och svagheter.

Redogörelse för användning av AI-verktyg

Under framtagandet av denna vetenskapliga rapport har generativa AI-verktyg (stora språkmodeller) använts som ett stöd i skrivprocessen. Verktygen har primärt nyttjats för att strukturera texten utifrån högskolans rekommenderade mall, för språklig korrekturläsning, samt som ett interaktivt bollplank för att formulera akademiska rubriker. All kod för den underliggande artefakten, liksom den konceptuella analysen, litteratururvalet och de slutgiltiga slutsatserna är författarnas eget originella arbete.

4 Genomförande

4.1 Metoder

För att utveckla det agentiska arbetsflödet konstruerades ett system i Python som hanterar inläsning, bearbetning och utvärdering av supportärenden (tickets). Kärnan i systemet utgörs av en LLM-agent driven av modellen llama 3.1:8b via ChatOllama, inställd med en temperatur på 0 för att maximera deterministiska och konsekventa svar.

Utvecklingsarbetet präglades av en medvetet avskalad designfilosofi, där målsättningen var att uppnå ett fungerande system med minsta möjliga tekniska komplexitet. Istället för komplicerad kod designades arkitekturen utifrån strategin att begränsa modellens frihetsgrader genom enkla, kontrollerade informationsflöden och stegvis exekvering. Följande metoder tillämpades:

- **Atomära prompter (Micro-steps):** Istället för att låta modellen hantera hela ärendet i en enda prompt, implementerades en iterativ loop. I varje steg tvingas agenten fatta ett atomärt, avgränsat beslut: antingen använda ett specifikt verktyg (tool) eller generera ett slutgiltigt svar (final). Detta minskar risken för hallucinationer och gör beslutsprocessen transparent.

- **Isolerad kontext:** Vid varje iteration får modellen en uppdaterad state prompt. Denna prompt säkerställer att modellen enbart presenteras för exakt den information som behövs i stunden: ärendets innehåll, aktuell historik (trajectory) och tillgängliga verktyg. Trajectory-loggen fungerar som agentens korttidsminne och säkerställer att den inte upprepar tidigare verktygsanrop.

- **Read-Only Prompts för extern kunskap:** Systemet utrustades med lokala verktyg, varav ett är read_policy. Genom detta verktyg kan agenten dynamiskt läsa in domänspecifik kunskap (exempelvis retur- eller faktureringspolicyer). Dessa dokument injiceras i kontexten som statisk, skrivskyddad information (Read-Only), vilket förhindrar modellen från att förvanska affärsreglerna under exekveringen.

- **Kontrollerad utdata (Parsing):** Modellens svar tvingas in i ett fördefinierat JSON-format. En underliggande funktion, parse_decision, fångar upp och avbryter processen säkert om modellen avviker från det förväntade formatet.

4.2 Test

Testningen av artefakten genomfördes genom att låta systemet bearbeta 12 fördefinierade supportärenden (tickets) från filen data/tickets.json. Testmiljön kunde köras på två sätt. Först användes app/main.py för att demonstrera det agentiska arbetsflödet genom funktionen run_agent(ticket). Därefter användes eval/run_eval.py för att jämföra det agentiska arbetsflödet med en enkel regelbaserad baseline. Baseline-flödet, implementerat i eval/baseline.py, utförde endast direkt routing baserat på ärendetyp och enkla fältkontroller. Det använde inte språkmodell, policyhämtning, verktygsanrop, trajectory eller iterativt beslutsfattande. Syftet med baseline var därför inte att vara en avancerad lösning, utan att fungera som en enkel jämförelsepunkt mot det agentiska arbetsflödet. Evaluation-runnern körde båda flödena mot samma supportärenden. För varje ärende sparades final answer, antal

registrerade steg, antal verktygsanrop, eventuella validatoranmärkningar och om ärendet passerade utan guardrail-flaggning. Resultatet skrevs till eval/evaluation_results.json, vilket gjorde testkörningen reproducerbar och möjlig att granska i efterhand.

4.3 Resultat

Resultaten från evaluation-körningen visar att både baseline och det agentiska arbetsflödet behandlade samtliga 12 supportärenden. Båda flödena hade 7 ärenden utan validatoranmärkning och 5 ärenden som flaggades av den mekaniska vakten. Dessa fem flaggningar ska inte förstås som tekniska körfel, utan som ärenden där affärsreglerna indikerade risk, saknad information eller behov av manuell granskning. Exempel på sådana fall var returer utanför 30-dagarsfönstret, faktura- eller returärenden utan ordernummer samt retur av använd personlig utrustning.

Mått	Baseline	Agentiskt workflow
Antal testade ärenden	12	12
Ärenden utan validatoranmärkning	7	7
Ärenden flaggade av guardrail	5	5
Andel utan validatoranmärkning	0,58	0,58
Genomsnittliga verktygsanrop	0,0	1,5
Genomsnittliga registrerade steg	1,0	4,0

Tabellen visar att det agentiska arbetsflödet inte uppnådde en högre andel ärenden utan validatoranmärkning än baseline i denna begränsade testmängd. Skillnaden låg istället i processens struktur. Baseline utförde en enkel enstegsbaserad routing utan verktygsanrop. Det agentiska arbetsflödet använde däremot i genomsnitt 1,5 verktygsanrop och 4,0 registrerade steg per ärende. Detta visar att artefakten uppfyller centrala agentiska krav såsom verktygsanvändning, iterativt beslutsfattande, state/trajectory och efterföljande validering. Ett exempel på skillnaden kan ses i faktura- och returärenden. Baseline returnerade en enkel routingtext baserad på ärendetyp, medan det agentiska arbetsflödet först kontrollerade obligatoriska fält och vid behov läste relevant policy innan ett slutligt svar genererades. Vid ärenden med saknad information, exempelvis ordernummer, kunde agenten avbryta processen och meddela att ärendet inte kunde triageras förrän informationen kompletterats.

4.4 Validering

För att säkerställa att artefakten inte enbart förlitar sig på språkmodellens interna logik, implementerades en frikopplad valideringsfas. Efter att agenten har returnerat sitt slutgiltiga

beslut (final answer), skickas hela kontexten till en separat kontrollfunktion (validate_decision).

Denna funktion agerar som en deterministisk, guardrail (Mechanical Guard). Dess primära syfte är att inspektera det agentiska beslutet mot verksamhetens oföränderliga affärsregler. Exempel på regler som valideras är:

- Att returärenden som överstiger tidsfönstret på 30 dagar flaggas för manuell granskning.
- Att faktura- och returärenden de facto innehåller ett giltigt ordernummer innan de dirigeras vidare till supporten.

Genom att separera agentens flexibla texttolkning från den strikta regelvalideringen skapas ett mer kontrollerbart arbetsflöde. Mechanical Guard fungerar som ett sista kontrollsteg som flaggar beslut som bryter mot affärsregler eller kräver komplettering. I evaluation-resultatet flaggades 5 av 12 ärenden, bland annat på grund av utgångna returfönster, saknade ordernummer och retur av använd personlig utrustning. Detta visar att valideringen inte endast fungerar som ett tekniskt eftersteg, utan som en central säkerhetsmekanism i arbetsflödet.

4.5 Måluppfyllelse

Det övergripande målet för utvecklingsarbetet var att skapa en fungerande och tillförlitlig agentisk artefakt för automatiserad triagering av supportärenden. Artefakten uppfyller detta mål genom att kombinera atomära prompter, isolerad kontexthantering via trajectory, lokala verktyg för policyhämtning och fältkontroll samt en oberoende mekanisk vakt för regelvalidering. Evaluation-resultatet visar att det agentiska arbetsflödet inte hade högre validator-pass rate än baseline i den begränsade testmängden. Däremot demonstrerade artefakten ett tydligt agentiskt beteende genom fler registrerade steg och faktisk verktygsanvändning. Baseline hade i genomsnitt 0,0 verktygsanrop och 1,0 steg per ärende, medan det agentiska arbetsflödet hade 1,5 verktygsanrop och 4,0 steg per ärende. Måluppfyllelsen ligger därför främst i att artefakten visar ett reproducerbart, spårbart och validerat agentiskt arbetsflöde, snarare än att den i denna testmängd uppvisar högre automatisk träffsäkerhet än baseline.

5 Analys

I detta kapitel analyseras framtagandet av artefakten och de testresultat som redovisades i föregående kapitel. Evaluation-resultatet visar att baseline och det agentiska arbetsflödet hade samma andel ärenden utan validatoranmärkning, 7 av 12.

Detta innebär att resultaten inte bör tolkas som att det agentiska arbetsflödet gav högre automatisk träffsäkerhet än baseline i den begränsade testmängden. Däremot visar loggarna att det agentiska arbetsflödet uppfyllde de centrala agentiska designmålen. Baseline arbetade i ett enda steg och använde inga verktyg.

Det agentiska arbetsflödet använde däremot i genomsnitt 4,0 registrerade steg och 1,5 verktygsanrop per ärende. Detta visar att systemet inte endast genererade ett direkt svar, utan arbetade genom en iterativ process där modellen kunde kontrollera obligatoriska fält, läsa policyinformation och därefter formulera ett slutligt beslut. Trajectory-loggen blev därmed viktig både som korttidsminne och som spårbar dokumentation av agentens beslutsprocess.

Analysen visar även att den mekaniska vakten fyllde en central funktion. De 5 validatorflaggade ärendena representerade inte tekniska fel, utan situationer där affärsreglerna krävde stopp, komplettering eller manuell granskning. Exempel på detta var returer utanför 30-dagarsfönstret, saknade ordernummer och retur av använd personlig utrustning.

Detta stärker argumentet för att LLM-baserade system inte bör ges ensam beslutanderätt i regelstyrda processer. Den mest relevanta nyttan med arkitekturen är därför ökad kontroll, spårbarhet och möjlighet att separera flexibel texttolkning från deterministisk regelvalidering.

6 Diskussion och slutsatser

Resultatet av utvecklingsarbetet visar att ett agentiskt arbetsflöde kan användas för att strukturera och kontrollera triagering av supportärenden i en avgränsad testmiljö. Artefakten visar inte att agenten i denna testmängd uppnår högre validator-pass rate än en enkel baseline, eftersom båda flödena hade 7 ärenden utan validatoranmärkning och 5 guardrail-flaggade ärenden.

Däremot visar resultaten att det agentiska arbetsflödet erbjuder en mer spårbar och kontrollerad process genom verktygsanvändning, policyhämtning, trajectory och efterföljande validering. Den praktiska implikationen är att e-handelsföretag potentiellt kan använda agentiska arbetsflöden för att avlasta kundtjänst, men att sådana system bör kompletteras med deterministiska kontrollmekanismer och mänsklig granskning vid riskärenden.

En begränsning i arbetet är att systemet enbart har testats på en fördefinierad, välstrukturerad mängd JSON-data. Verkliga supportärenden innehåller ofta mer ostrukturerat språk, sarkasm, flera problem i samma meddelande eller ofullständig information, vilket skulle kräva mer omfattande tester innan systemet kan bedömas i en produktionsliknande miljö.

Slutsatser

För att besvara studiens övergripande fråga om hur ett agentiskt arbetsflöde kan utformas för tillförlitlig triagering och hantering av supportärenden har två delfrågor utvärderats:

1. **Hur påverkar användningen av atomära prompter agentens förmåga att utföra korrekta verktygsval?**

   Resultatet visar att atomära prompter bidrog till att skapa ett spårbart och stegvis beslutsflöde. Agenten tvingades i varje iteration välja mellan att använda ett verktyg eller lämna ett slutligt svar, vilket gjorde processen mer kontrollerbar än ett vanligt enstegssvar. Evaluation-resultatet visar dock inte att detta gav högre validator-pass rate än baseline i den begränsade testmängden. Den tydligaste effekten var istället att agenten faktiskt använde verktyg och arbetade iterativt: i genomsnitt 1,5 verktygsanrop och 4,0 registrerade steg per ärende. Detta stödjer slutsatsen att atomära prompter är användbara för att operationalisera agentiskt beteende och göra beslutsprocessen mer transparent.

2. **Vilken roll spelar mekaniska vakter (validering) för att säkerställa att LLM-agenten följer uppsatta affärsregler?**

   Mekaniska vakter spelade en central roll för att säkerställa att ärenden som bryter mot affärsregler eller kräver komplettering inte passerar utan anmärkning. I evaluation-resultatet flaggades 5 av 12 ärenden av validate_decision. Dessa flaggningar gällde bland annat returer utanför 30-dagarsfönstret, saknade ordernummer och retur av använd personlig utrustning. Detta visar att den mekaniska vakten inte enbart är ett tekniskt tillägg, utan en viktig del av systemets kontrollstruktur. Samtidigt visar resultatet att valideringen behöver förstås som ett säkerhetslager snarare än som ett mått på att agenten alltid löser ärendet bättre än baseline. Eftersom både baseline och agentiskt workflow fick samma antal validatoranmärkningar i testet är den huvudsakliga slutsatsen att guardrails bidrar till kontroll och riskidentifiering, medan det agentiska arbetsflödet bidrar med spårbarhet, verktygsanvändning och strukturerad hantering före valideringssteget.

Sammanfattningsvis visar artefakten att ett tillförlitligt agentiskt arbetsflöde bör kombinera språkmodellens textförståelse med traditionella programvarustrukturer: tydlig stegvis styrning, explicit verktygsanvändning, trajectory-loggning och oberoende validering. I denna begränsade testmiljö låg styrkan inte i högre validator-pass rate än baseline, utan i att arbetsflödet blev mer transparent, reproducerbart och kontrollerbart.

7 Referenser

Elizabeth, M. *m.fl.* (2025) ”Exploring ReAct Prompting for Task-Oriented Dialogue: Insights and Shortcomings”. arXiv. Tillgänglig vid: https://doi.org/10.48550/arXiv.2412.01262.

Rebedea, T. *m.fl.* (2023) ”NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails”. arXiv. Tillgänglig vid: https://doi.org/10.48550/arXiv.2310.10501.

Schick, T. *m.fl.* (2023) ”Toolformer: Language Models Can Teach Themselves to Use Tools”. arXiv. Tillgänglig vid: https://doi.org/10.48550/arXiv.2302.04761.

Shi, F. *m.fl.* (2023) ”Large Language Models Can Be Easily Distracted by Irrelevant Context”. arXiv. Tillgänglig vid: https://doi.org/10.48550/arXiv.2302.00093.

UNIVERSITY
OF BORÅS

Besöksadress: Allégatan 1 · Postadress: 501 90 Borås · Tfn: 033-435 40 00 · E-post: registrator@hb.se · Webb: www.hb.se